

simple_fuzzer 测试报告

1. 测试目标

本次测试用于验证 `simple_fuzzer` 在不同调度策略下的 fuzzing 效果，重点关注以下要求：

- 1. 对当前项目中实现的两个调度策略 `path` 与 `density` 分别进行测试。
- 2. 对每个调度策略分别测试 60 秒和 300 秒两种运行时间。
- 3. 记录并比较覆盖率、唯一崩溃数、执行次数和长时间运行稳定性。

2. 测试环境与命令

测试入口为 README 中说明的 `main.py`，通过 `uv run` 执行。

基础检查命令：

```
uv run main.py --help
uv run python -m compileall .
```

测试矩阵如下：

```
uv run main.py --sample 1 --run-time 60 --quiet --schedule path
uv run main.py --sample 2 --run-time 60 --quiet --schedule path
uv run main.py --sample 3 --run-time 60 --quiet --schedule path
uv run main.py --sample 4 --run-time 60 --quiet --schedule path

uv run main.py --sample 1 --run-time 300 --quiet --schedule path
uv run main.py --sample 2 --run-time 300 --quiet --schedule path
uv run main.py --sample 3 --run-time 300 --quiet --schedule path
uv run main.py --sample 4 --run-time 300 --quiet --schedule path

uv run main.py --sample 1 --run-time 60 --quiet --schedule density
uv run main.py --sample 2 --run-time 60 --quiet --schedule density
uv run main.py --sample 3 --run-time 60 --quiet --schedule density
uv run main.py --sample 4 --run-time 60 --quiet --schedule density

uv run main.py --sample 1 --run-time 300 --quiet --schedule density
uv run main.py --sample 2 --run-time 300 --quiet --schedule density
uv run main.py --sample 3 --run-time 300 --quiet --schedule density
uv run main.py --sample 4 --run-time 300 --quiet --schedule density
```

为了避免不同测试之间互相覆盖结果，每组测试均使用独立的 `_result_matrix_*` 与 `_persist_matrix_*` 目录保存结果和 checkpoint。

3. 覆盖率统计口径

本报告中的覆盖率采用“目标 Sample 函数自身的可执行语句行覆盖率”作为统计口径。

也就是说：

- Sample 1 只统计 `sample1()` 中的可执行语句。
- Sample 2 统计 `sample2()` 及其内部函数 `can_convert_to_int()`。
- Sample 3 只统计 `sample3()` 中的可执行语句。
- Sample 4 只统计 `sample4()` 中的可执行语句。

结果文件中的 `covered_line` 会包含 `Coverage` 工具函数、`trace()`、以及 Sample 4 中 `HTMLParser` 标准库内部执行路径。为了避免这些辅助代码和标准库代码影响实验结论，覆盖率计算时不将它们纳入目标函数覆盖率分母。

4. Scheduler 实现说明

4.1 PathPowerSchedule

`PathPowerSchedule` 基于路径频率分配能量。路径出现次数越少，对应 seed 的能量越高，从而鼓励 fuzzer 继续探索较少出现的路径。

4.2 DensityPowerSchedule

原始 `DensityPowerSchedule` 只根据覆盖行数和输入长度计算能量，逻辑较简单，且缺少 `PathGreyBoxFuzzer` 所需的 `path_frequency` 和 `update_path_frequency()` 接口。

本次测试前已将其改进为：

- 维护 `path_frequency`，记录路径出现频率。
- 维护 `line_frequency`，记录覆盖行出现频率。
- 按“稀有覆盖行分数 / 输入长度 / 路径频率”分配 seed 能量。
- 与 `PathGreyBoxFuzzer` 的路径统计接口兼容。

此外，`GreyBoxFuzzer` 的种子入池策略也进行了调整：只要输入带来新覆盖，即使该输入触发 crash，也会加入 population，作为后续变异的 frontier seed。该策略对 Sample 3 这类深层条件分支尤其重要，因为部分更深覆盖会先以异常形式暴露。

5. 测试结果

5.1 目标函数覆盖率

Scheduler	运行时间	Sample 1	Sample 2	Sample 3	Sample 4
path	60s	8/8 (100.00%)	13/13 (100.00%)	9/9 (100.00%)	2/2 (100.00%)
path	300s	8/8 (100.00%)	13/13 (100.00%)	9/9 (100.00%)	2/2 (100.00%)
density	60s	8/8 (100.00%)	13/13 (100.00%)	9/9 (100.00%)	2/2 (100.00%)
density	300s	8/8 (100.00%)	13/13 (100.00%)	9/9 (100.00%)	2/2 (100.00%)

5.2 唯一崩溃数量

Scheduler	运行时间	Sample 1	Sample 2	Sample 3	Sample 4
path	60s	6	4	9	0
path	300s	6	4	9	0
density	60s	6	4	9	0
density	300s	6	4	9	0

5.3 执行次数

Scheduler	运行时间	Sample 1	Sample 2	Sample 3	Sample 4
path	60s	12,721	527,842	778,322	43,358
path	300s	60,557	2,323,014	3,496,138	218,977
density	60s	8,108	452,145	676,354	34,756
density	300s	40,660	2,683,371	3,191,463	156,901

5.4 结果文件

测试结果保存在以下目录：

```
_result_matrix_path_60/  
_result_matrix_path_300/  
_result_matrix_density_60/  
_result_matrix_density_300/
```

每个目录下包含：

```
Sample-1.pkl  
Sample-2.pkl  
Sample-3.pkl  
Sample-4.pkl
```

对应的 checkpoint、crash 和 coverage snapshot 保存在：

```
_persist_matrix_path_60_sample_*/  
_persist_matrix_path_300_sample_*/  
_persist_matrix_density_60_sample_*/  
_persist_matrix_density_300_sample_*/
```

6. 结果分析

6.1 覆盖率要求满足情况

从覆盖率结果看，两个调度策略在 60 秒和 300 秒测试中均能够在 4 个 Sample 上达到 100% 的目标函数行覆盖率。

6.2 60 秒与 300 秒对比

300 秒测试相比 60 秒测试显著增加了执行次数。例如：

- **path** + Sample 2 从 527,842 次增加到 2,323,014 次。
- **density** + Sample 2 从 452,145 次增加到 2,683,371 次。
- **path** + Sample 3 从 778,322 次增加到 3,496,138 次。

但是目标函数覆盖率和唯一崩溃数量没有继续增加。这说明在当前 4 个 Sample 上，主要分支和崩溃类型都能在 60 秒内被发现，继续延长到 300 秒主要增加执行量和稳定性验证，对目标函数覆盖率的边际提升较小。

Sample 4 的结果也体现了这一点。虽然 Sample 4 的目标函数只有两行，因此目标函数覆盖率很快达到 100%，但其内部调用了 Python 标准库 **HTMLParser**，因此原始 **covered_line** 中会记录大量标准库内部路径。300 秒测试可以继续探索更多 HTMLParser 内部路径，但这不影响目标 Sample 函数本身的覆盖率结论。

6.3 Path 与 Density 对比

path 调度策略更直接地根据路径频率选择 seed，适合鼓励较少出现路径的继续探索。

density 调度策略在改进后同时考虑：

- seed 覆盖的行是否稀有；
- seed 触发的路径是否稀有；
- seed 输入长度是否较短。

从测试结果看，两种策略最终都能达到 100% 目标覆盖率。执行次数上，两者存在差异：

- 在 Sample 1、Sample 3 和 Sample 4 上，**path** 的执行次数通常更高。
- 在 Sample 2 的 300 秒测试中，**density** 的执行次数更高。

造成差异的原因主要是两种调度策略对 seed 的选择偏好不同，进而影响输入长度、变异成本、目标函数异常路径和执行速度。由于 fuzzing 本身包含随机变异，不同轮次之间也会存在一定随机波动。

6.4 Crash 发现情况

Sample 1、Sample 2 和 Sample 3 均发现了多个唯一崩溃，Sample 4 未发现崩溃：

- Sample 1：6 类唯一崩溃。
- Sample 2：4 类唯一崩溃。
- Sample 3：9 类唯一崩溃。
- Sample 4：0 类唯一崩溃。

300 秒测试没有比 60 秒发现更多唯一崩溃，说明当前崩溃类型在 60 秒内已经基本饱和。Sample 4 未发现崩溃符合预期，因为其目标函数主要是调用标准库 HTMLParser 解析输入，该库对异常输入具有较强容错性。

7. 结论

本次测试完成了 **path** 和 **density** 两个 Scheduler 在 4 个 Sample 上的 60 秒与 300 秒完整测试。

测试结果表明：

1. 两个 Scheduler 在所有 Sample 上均达到 100% 目标函数行覆盖率。
2. 300 秒测试相比 60 秒测试显著增加执行次数，但覆盖率和唯一崩溃数量基本不再增长，说明当前 Sample 的覆盖和崩溃发现已在 60 秒内基本饱和。
3. 改进后的 **DensityPowerSchedule** 能够正常作为独立调度策略运行，并取得与 **PathPowerSchedule** 同级别的覆盖效果。
4. Fuzzer 在长时间运行下可以正常保存结果、checkpoint、crash 信息和 coverage snapshot，未出现框架层面的运行错误。

综上，当前实现满足实验测试要求，并具备较好的稳定性和可解释性。